

Lab 10 : Bankers algorithm

1. Write a program in C to implement Banker's Algorithm.

code:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int r, n, k, sum = 0, c = 0, x = 0, y = 0;
    printf("Enter the total no of processes: ");
    scanf("%d", &n);
    printf("Enter the total no of resources: ");
    scanf("%d", &r);

    int *total = malloc(r * sizeof(int));
    int *w = malloc(r * sizeof(int));
    int *s = malloc(n * sizeof(int));
    int *flag = malloc(n * sizeof(int));

    int **all = malloc(n * sizeof(int *));
    int **max = malloc(n * sizeof(int *));
    int **rneed = malloc(n * sizeof(int *));
    int **work = malloc(n * sizeof(int *));

    for (int i = 0; i < n; i++) {
        all[i] = malloc(r * sizeof(int));
        max[i] = malloc(r * sizeof(int));
        rneed[i] = malloc(r * sizeof(int));
        work[i] = malloc(r * sizeof(int));
    }

    for (int i = 0; i < n; i++) {
        printf("\nEnter the details for process P%d\n", i);
        printf("Enter the allocation: ");
        for (int j = 0; j < r; j++) {
            scanf("%d", &all[i][j]);
        }
        printf("Enter Max: ");
        for (int j = 0; j < r; j++) {
            scanf("%d", &max[i][j]);
        }
        flag[i] = 0;
    }

    printf("\nEnter the total resources: ");
    for (int i = 0; i < r; i++) {
        scanf("%d", &total[i]);
    }

    for (int i = 0; i < r; i++) {
        for (int j = 0; j < n; j++) {
            sum += all[j][i];
        }
    }
}
```

```

    }
    w[i] = total[i] - sum;
    sum = 0;
}

for (int i = 0; i < n; i++) {
    for (int j = 0; j < r; j++) {
        rneed[i][j] = max[i][j] - all[i][j];
    }
}

for (int g = 0; g < n; g++) {
    while (flag[g] != 1) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < r; j++) {
                if (flag[i] != 1) {
                    if (rneed[i][j] <= w[j]) {
                        c++;
                    }
                }
            }
            if (c == r) {
                s[x] = i;
                x++;
                k = 0;
                while (k != r) {
                    w[k] += all[i][k];
                    work[y][k] = w[k];
                    k++;
                }
                y++;
                flag[i] = 1;
                c = 0;
            } else {
                c = 0;
            }
        }
    }
}

printf("\n\nThe Safe Sequence is --> ");
for (int i = 0; i < n; i++) {
    printf("P%d ", s[i]);
}

printf("\n\nProcess\t\tAllocation\tMax\t\tWork\t\tNeed\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t\t", i);
    for (int j = 0; j < r; j++) {
        printf("%d ", all[i][j]);
    }
    printf("\t\t");
    for (int j = 0; j < r; j++) {
        printf("%d ", max[i][j]);
    }
}

```

```

        printf("\t\t");
        for (int j = 0; j < r; j++) {
            printf("%d ", work[i][j]);
        }
        printf("\t\t");
        for (int j = 0; j < r; j++) {
            printf("%d ", rneed[i][j]);
        }
        printf("\n");
    }

    // Free allocated memory
    for (int i = 0; i < n; i++) {
        free(all[i]);
        free(max[i]);
        free(rneed[i]);
        free(work[i]);
    }
    free(all);
    free(max);
    free(rneed);
    free(work);
    free(total);
    free(w);
    free(s);
    free(flag);

    return 0;
}

```

output:

```

PS C:\Users\samir\Desktop\project> cd "c:\Users\samir\Desktop\project\" ; if ($?) { gcc lab6.c -o lab6 } ; if ($?) { .\lab6 }
Enter the total no of processes: 5
Enter the total no of resources: 3

Enter the details for process P0
Enter the allocation: 0 1 0
Enter Max: 7 5 3

Enter the details for process P1
Enter the allocation: 2 0 0
Enter Max: 3 2 2

Enter the details for process P2
Enter the allocation: 3 0 2
Enter Max: 9 0 2

Enter the details for process P3
Enter the allocation: 2 1 1
Enter Max: 4 2 2

Enter the details for process P4
Enter the allocation: 0 0 2
Enter Max: 5 3 3

Enter the total resources: 10 5 7

The Safe Sequence is --> P1 P3 P4 P0 P2

Process    Allocation    Max        Work        Need
P0         0 1 0        7 5 3      5 3 2      7 4 3
P1         2 0 0        3 2 2      7 4 3      1 2 2
P2         3 0 2        9 0 2      7 4 5      6 0 0
P3         2 1 1        4 2 2      7 5 5      2 1 1
P4         0 0 2        5 3 3      10 5 7     5 3 1
PS C:\Users\samir\Desktop\project>

```